Task Assignment

AlgoExpert · Medium · Greedy Algorithms

The problem

- 2k tasks with durations, k workers doing two tasks each in parallel; minimise the longest worker time.
- Pair shortest with longest.
- Returns index pairs.

Intuition

- Pair the shortest task with the longest so no worker gets two long tasks.

Brute force vs optimal

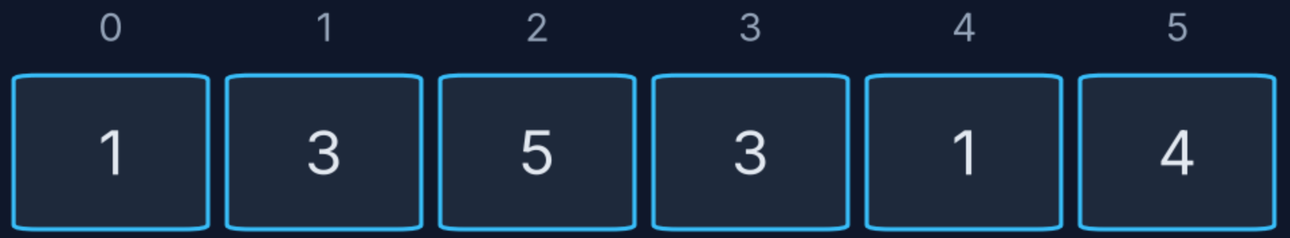
- Brute force: try every pairing → (2k)! / (2^k · k!)
- Greedy: sort, pair ends → O(n log n)
- Exchange argument proves optimality

Approach

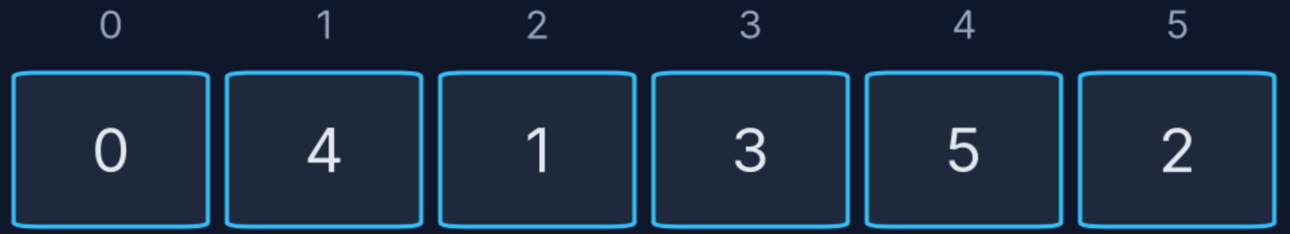
- 1. Sort task indices by duration
- 2. Pair the ends moving inward
- 3. Return the index pairs

Worked example

task durations



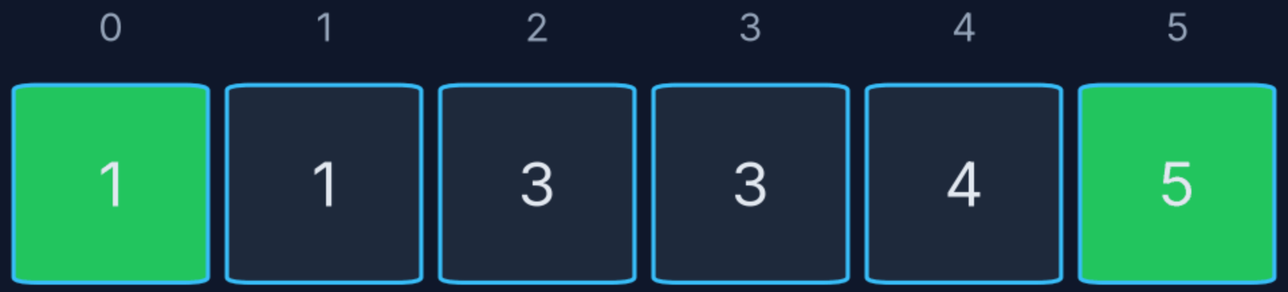
indices sorted by duration



Sort task indices by duration

Worked example

sorted durations

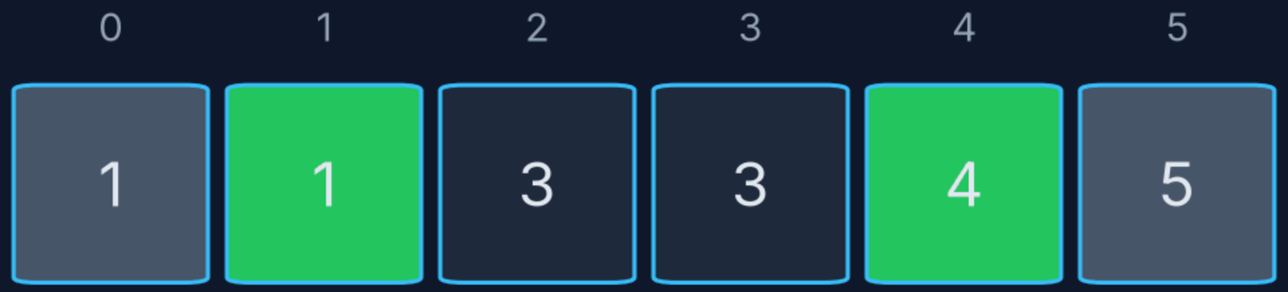



```
worker 1: tasks 0 & 2 → 1 + 5 = 6
```

Pair shortest with longest: 1 + 5

Worked example

sorted durations

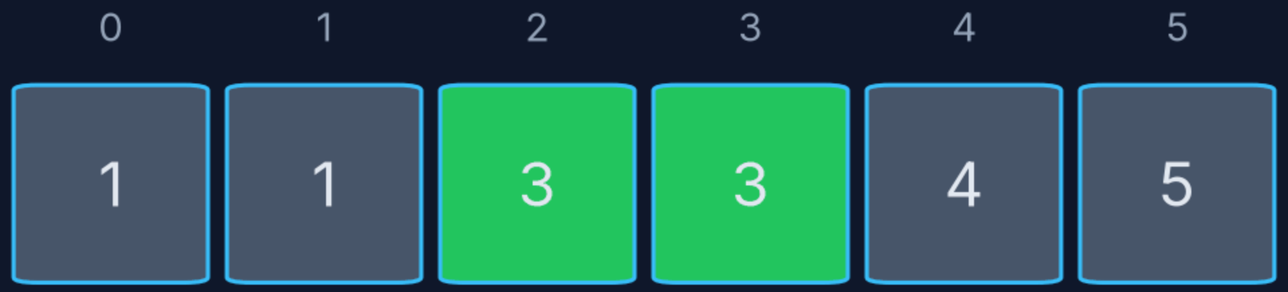



```
worker 2: tasks 4 & 5 → 1 + 4 = 5
```

Pair shortest with longest: 1 + 4

Worked example

sorted durations




```
worker 3: tasks 1 & 3 → 3 + 3 = 6
```

Pair shortest with longest: 3 + 3

Java solution

```java
public class TaskAssignment {
    public static List<List<Integer>> taskAssignment(int k, List<Integer> tasks) {
        List<Integer> idx = new ArrayList<>();
        for (int i = 0; i < tasks.size(); i++) idx.add(i);
        idx.sort(Comparator.comparingInt(tasks::get));
        List<List<Integer>> out = new ArrayList<>();
        for (int i = 0; i < k; i++) out.add(List.of(idx.get(i), idx.get(idx.size() - 1 - i)));
        return out;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Edge cases & pitfalls

- Keep original indices (sort indices, not values)
- Duplicate durations
- Exactly 2k tasks

Interview tips & follow-ups

- Sketch the exchange argument
- Follow-up: workers take more than two tasks (harder, partition-like)

Recap

- Pair the shortest task with the longest so no worker gets two long tasks.
- Complexity: O(n log n)